Lecture Note: Data Structures – Lists

Nishut Suman 7 Jan, 2026 At 11:00 AM Notes Foundation Recommended Resource

Associated Lectures (1)

Description Discussions

Lecture Note: List in Python

In Python, a **list** is a built-in data structure that can hold an ordered collection of items. Unlike arrays in some languages, Python lists are highly flexible.

Key Features of Lists

- **Can contain duplicate items**
- **Mutable:** items can be modified, replaced, or removed
- **Ordered:** maintains the order in which items are added
- **Index-based:** items are accessed using their position (starting from 0)
- **Can store mixed data types** (integers, strings, booleans, even other lists)

Creating a List

Lists can be created in several ways, such as using square brackets, the list() constructor or by repeating elements. Let's look at each method one by one with example:

1. Using Square Brackets

We use square brackets [] to create a list directly.

```
a = [1, 2, 3, 4, 5]        # List of integers
b = ['India', 'Russia', 'China']   # List of strings
c = [1, 'hello', 3.14, True]       # Mixed data types
print(a)
print(b)
print(c)
```

Output:

```
[1, 2, 3, 4, 5]
['India', 'Russia', 'China']
[1, 'hello', 3.14, True]
```

---

2. Using list() Constructor

We can also create a list by passing an iterable (like a tuple, string or another list) to the list() function.

```
a = list((1, 2, 3, 'Happy_Diwali', 4.5))
print(a)
b = list("GFG")
print(b)
```

Output:

```
[1, 2, 3, 'Happy_Diwali', 4.5]
['G', 'F', 'G']
```

---

3. Creating List with Repeated Elements

We can use the multiplication operator * to create a list with repeated items.

```
a = [0] * 5
print(a)
```

Output:

```
[0, 0, 0, 0, 0]
```

Accessing List Elements

Elements in a list are accessed using indexing. Python indexes start at 0, so a[0] gives the first element. Negative indexes allow access from the end (e.g., -1 gives the last element).

```
a = [10, 20, 30, 40, 50]
print(a[0])      # First element
print(a[-1])     # Last element
```

Output:

```
10
50
```

Adding Elements to a List

- **append()** → Adds an element at the end of the list
- **extend()** → Adds multiple elements to the end of the list
- **insert()** → Adds an element at a specific position
- **clear()** → Removes all items from the list

```
a = []
a.append(10)
print("After append(10):", a)
a.insert(0, 5)
print("After insert(0, 5):", a)
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
a.clear()
print("After clear():", a)
```

Output:

```
After append(10): [10]
After insert(0, 5): [5, 10]
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
After clear(): []
```

Updating Elements in a List

Since lists are **mutable**, we can update elements by accessing them via their index.

```
a = [10, 20, 30, 40, 50]
a[1] = 25
print(a)
```

Output:

```
[10, 25, 30, 40, 50]
```

Removing Elements from a List

- **remove()** → Removes the first occurrence of an element
- **pop()** → Removes the element at a specific index (or last if no index is specified)
- **del statement** → Deletes an element at a specified index

```
a = [10, 20, 30, 40, 50]
a.remove(30)
print("After remove(30):", a)
popped_val = a.pop(1)
print("Popped element:", popped_val)
print("After pop(1):", a)
del a[0]
print("After del a[0]:", a)
```

Output:

```
After remove(30): [10, 20, 40, 50]
Popped element: 20
After pop(1): [10, 40, 50]
After del a[0]: [40, 50]
```

Indexing in Python

Indexing is the process of accessing an element in a sequence using its position in the sequence (its index). In Python, indexing starts from 0, which means the first element in a sequence is at position 0, the second element is at position 1, and so on.

- The first element is at position 0
- The second element is at position 1, and so on.

```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[0])  # Output: 'apple'
print(my_list[1])  # Output: 'banana'
```

Slicing in Python

Slicing is used to access a **sub-sequence** of elements from a list by specifying a starting and ending index. The syntax for slicing is:

```
sequence[start_index:end_index]
```

- Note- The end_index` is **exclusive**, meaning the element at that index is not included.

```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[1:3])  # Output: ['banana', 'cherry']
```

Iterating Over Lists

We can iterate over lists using loops — this is useful for performing actions on each item.

```
a = ['apple', 'banana', 'cherry']
for item in a:
    print(item)
```

Output:

```
apple
banana
cherry
```

List Comprehension

List comprehension is a **concise and elegant** way to create lists in a single line of code. It's useful for applying an operation or filter to items in an iterable (like a list or range).

```
squares = [x**2 for x in range(1, 6)]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

Explanation:

- for x in range(1, 6): loops through numbers from 1 to 5 (excluding 6).
- x**2: squares each number.
- [ ]: collects all squared numbers into a new list.

Summary:

- Lists are **ordered, mutable, and index-based**.
- You can **add, remove, or modify** items easily.
- Supports **slicing, iteration, and comprehensions** for flexible operations.

-